

Détection de Tweets Suspects

Projet — Construction de Modèles et Déploiement

M2 FD&IA

Juin 2026

Introduction

Les réseaux sociaux sont une source majeure d'information mais aussi un vecteur de diffusion de contenus suspects (haineux, offensants, trompeurs). Ce projet vise à développer une solution de classification automatique capable d'identifier ces tweets suspects.

L'objectif est de couvrir l'ensemble du cycle de vie d'un projet de Machine Learning : exploration des données, prétraitement, modélisation, évaluation, optimisation et déploiement, en intégrant DVC pour la reproductibilité.

Dataset

Le jeu de données provient de Google Drive et contient **60 000 tweets** avec une étiquette binaire :

- **0** : Non suspect
- **1** : Suspect

Deux variables sont disponibles : message (texte du tweet) et label (classe).

Méthodologie

Prétraitement

Le nettoyage du texte est effectué via `src/preprocessing.py` et comprend les étapes suivantes :

1. **Passage en minuscules** — uniformisation de la casse
2. **Suppression des URLs** — les liens n'apportent pas d'information sémantique
3. **Suppression des caractères spéciaux** — seuls les caractères alphabétiques sont conservés
4. **Suppression des stop words** — mots fréquents non discriminants (NLTK english stop words)
5. **Suppression des tweets vides** — 15 tweets vidés par le nettoyage sur 60 000

Ces choix sont documentés dans le notebook d'analyse exploratoire (`notebooks/01_eda.ipynb`). La modélisation complète est détaillée dans `notebooks/02_modeling.ipynb`.

Représentation des données

La représentation choisie est **TF-IDF** (Term Frequency — Inverse Document Frequency) avec un maximum de **5 000 features**. Ce choix se justifie par :

- Sa simplicité et son efficacité pour la classification de textes courts
- Sa capacité à pondérer l'importance des mots dans le corpus
- Sa compatibilité avec les modèles linéaires et naïfs bayésiens

Modèles utilisés

Trois algorithmes ont été implémentés et comparés :

1. Régression Logistique

Modèle linéaire avec `class_weight="balanced"` pour gérer le déséquilibre des classes. Limité à 1000 itérations pour assurer la convergence.

2. Naive Bayes (Multinomial)

Classifieur probabiliste adapté aux données de comptage (TF-IDF).
Performant sur les textes mais sensible à la corrélation entre features.

3. Random Forest

Ensemble de 100 arbres de décision avec `class_weight="balanced"`.
Capture les interactions non linéaires entre les mots.

Gestion du déséquilibre

La classe suspect est sous-représentée. La stratégie `class_weight="balanced"` ajuste les poids automatiquement. Une alternative (SMOTE) pourra être testée en optimisation.

Résultats

Performances des modèles

Modèle	Accuracy	Precision	Recall	F1-Score
Logistic Regression	96.07%	97.58%	98.06%	97.82%
Naive Bayes	92.50%	92.29%	99.99%	95.99%
Random Forest	97.09%	97.59%	99.21%	98.39%

Le **Random Forest** obtient les meilleures performances globales avec un F1-Score de **98.39%**. Naive Bayes a un recall parfait (99.99%) mais une précision plus faible.

Matrices de confusion

Les matrices de confusion sont disponibles dans `reports/figures/confusion_matrices.png`.

Courbes ROC et AUC

Les courbes ROC sont disponibles dans `reports/figures/roc_curves.png`. Les trois modèles présentent une AUC supérieure à 0.97, confirmant leur capacité de discrimination.

Validation croisée

La séparation train/test a été faite en **80/20 avec stratification** pour préserver la distribution des classes. Une validation croisée **5-fold** a été appliquée pour évaluer la stabilité des modèles (résultats dans `notebooks/02_modeling.ipynb`).

Pipeline DVC

Le pipeline reproductible se compose de 3 étapes :

`dvc repro`

1. **preprocess** — nettoyage du texte
2. **train** — entraînement des 3 modèles avec TF-IDF
3. **evaluate** — calcul des métriques et génération des graphiques

Le dataset est téléchargeable via `uv run python src/download.py` (téléchargement interactif depuis Google Drive).

Optimisation

Une recherche d'hyperparamètres (Grid Search) a été effectuée sur deux modèles :

- **Régression Logistique** : $c \in \{0.1, 1, 10\}$ — meilleur paramètre trouvé via validation croisée 3-fold
- **Random Forest** : $n_estimators \in \{50, 100\}$, $max_depth \in \{10, 20, None\}$ — meilleur paramètre trouvé via validation croisée 3-fold

Les résultats détaillés sont visibles dans `notebooks/02_modeling.ipynb`.

Déploiement

Deux solutions sont prévues :

- **Streamlit** : interface utilisateur pour saisir un tweet et obtenir la prédiction
- **API FastAPI** : endpoint REST pour la prédiction

Discussion

Limites

- La représentation TF-IDF ne capture pas le contexte sémantique
- Les modèles n'ont pas été testés sur des données réelles en streaming
- Pas de détection multilingue (stop words english uniquement)

Difficultés rencontrées

- Gestion des valeurs manquantes dans les tweets
- Déséquilibre des classes nécessitant des stratégies d'adaptation
- Mise en place du pipeline DVC avec téléchargement depuis Google Drive

Perspectives d'amélioration

- Utilisation de **Sentence Transformers** ou **BERT** pour une meilleure représentation
- Déploiement sur le cloud (Hugging Face Spaces)
- Dashboard de monitoring des performances en production
- Intégration CI/CD avec GitHub Actions
- Expérimentation avec MLflow pour le suivi des runs

Conclusion

Ce projet a permis de mettre en œuvre un pipeline complet de Machine Learning pour la détection de tweets suspects. Les meilleurs résultats ($F1 > 98\%$) montrent la faisabilité de la tâche avec des approches classiques. L'utilisation de DVC assure la reproductibilité du pipeline, et le déploiement via Streamlit/FastAPI permet une utilisation en conditions réelles.

Références

- Sujet : Détection_de_Tweet_Suspect_2026.md
- Code source : <https://github.com/...> (à compléter)
- Dataset : Google Drive